Defence Institute Of Advanced Technology (DIAT, PUNE)

DEPARTMENT OF APPIED MATHEMATICS

M.Sc in DATA SCIENCE

(2025-2027)

Project: SIMD ,ALGORITHUM AND ENVIRONMENT

Name – TITLI BISWAS & PRINCY PANDEY

Registration no – 25-60-03 & 25-60-11

Supervised by

Prof.(Dr.) Yogeshwar Singh Dadwhal

INTRODUCTION

Single Instruction Multiple Data (SIMD) is a parallel computing technique in which a single instruction is executed simultaneously on multiple data elements. It is an important concept from Flynn's Taxonomy and is widely used to improve computational speed and efficiency.

In traditional computing, operations are performed on one data item at a time, which can be slow for large datasets. SIMD overcomes this limitation by processing many data points in parallel, making it highly suitable for data-intensive tasks.

This approach is commonly used in areas such as image processing, scientific simulations, graphics rendering, and machine learning, where the same operation needs to be applied repeatedly to large amounts of data.

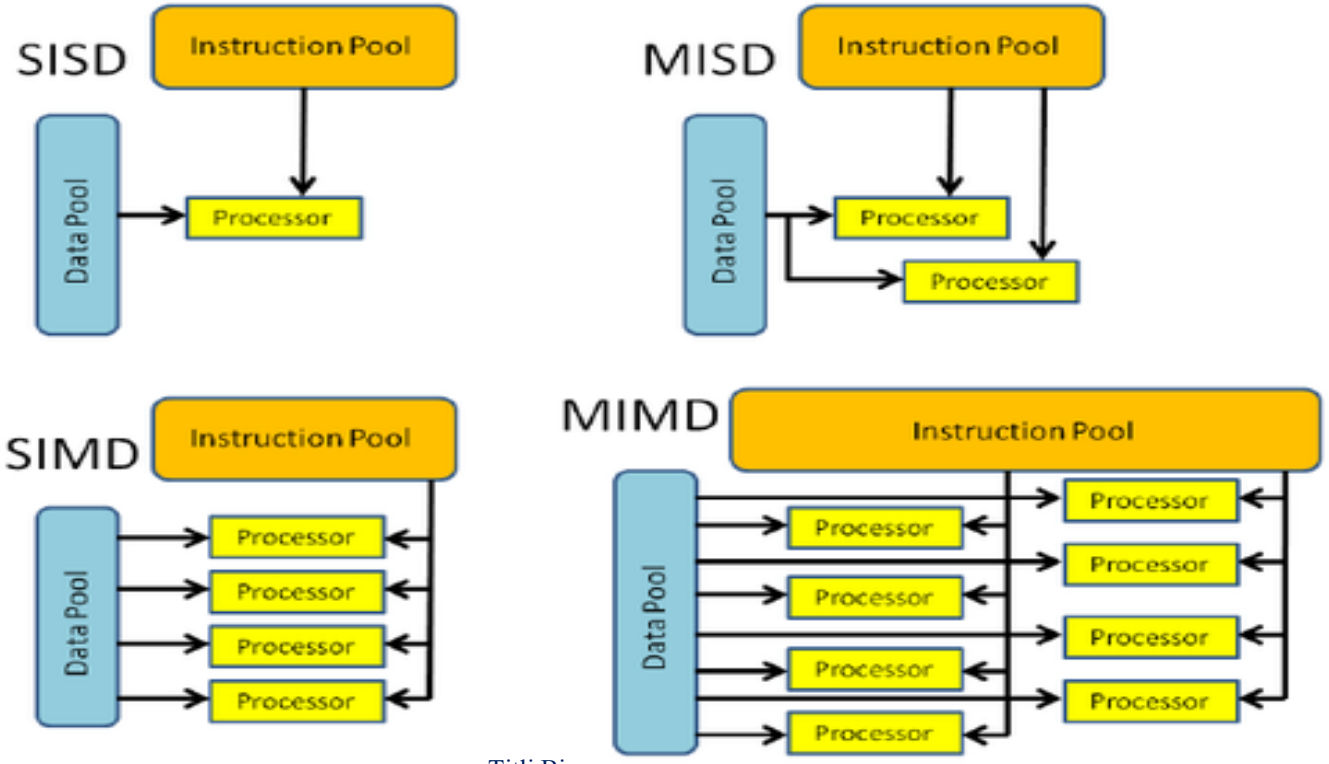
With the advancement of modern processors and parallel computing technologies, SIMD has become a fundamental technique for achieving high performance and optimizing resource utilization in today's computing systems.

FLYNN'S TAXONOMY

Flynn's taxonomy is a foundational classification scheme in computer architecture that organizes systems by how they handle instruction and data streams. Proposed by computer scientist **Michael J. Flynn** in 1966, it remains a core conceptual tool for describing parallelism in computing systems and guiding processor design.

Four-class model :

1. **SISD**
2. **SIMD**
3. **MISD**
4. **MIMD**



FLYNN'S TAXONOMY (Cont.)

Class	Description	Example Uses
SISD (Single Instruction, Single Data)	One processor executes one instruction on one data stream; traditional uniprocessors.	Early PCs, mainframes
SIMD (Single Instruction, Multiple Data)	One instruction operates on multiple data elements in parallel; efficient for vector/matrix operations.	Vector processors, GPUs
MISD (Multiple Instruction, Single Data)	Multiple instructions act on the same data stream; rare, used in specialized or fault-tolerant designs.	Certain control systems
MIMD (Multiple Instruction, Multiple Data)	Multiple processors independently execute different instructions on different data; basis of most multiprocessor and multicore systems.	Modern supercomputers, servers

SIMD ARCHITECTURE

SIMD systems consist of:

•**Control Unit (CU)** → Issues a single instruction

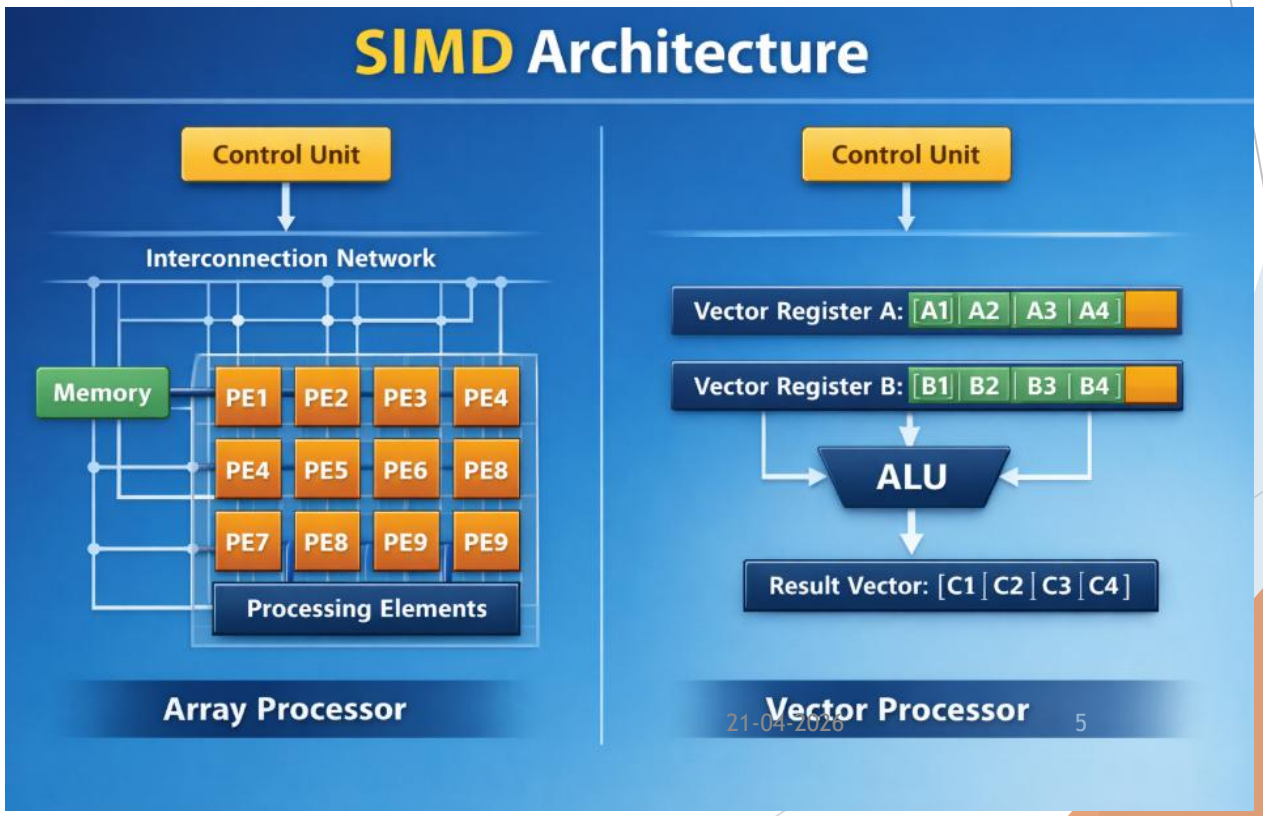
•**Processing Elements (PEs)** → Execute same instruction on different data

•**Memory Units** → Store vector data

❑ **Types of SIMD Architecture:**

1.Array Processor

2.Vector Processor



ARRAY PROCESSOR

An **Array Processor** is a type of SIMD architecture where **multiple processing elements (PEs) are arranged in a grid** and execute the **same instruction simultaneously on different data**.

❑ Main Components:

- **Control Unit (CU):** Sends a single instruction to all processing elements
- **Processing Elements (PEs):** Small processors arranged in array form (grid)
- **Memory Unit:** Supplies data to all PEs
- **Interconnection Network:** Connects CU, memory, and PEs for communication

❑ Working :

1. Control Unit broadcasts instruction
2. All PEs receive the same instruction
3. Each PE processes different data
4. Results are computed in parallel

➢ Example: Parallel addition of matrix elements

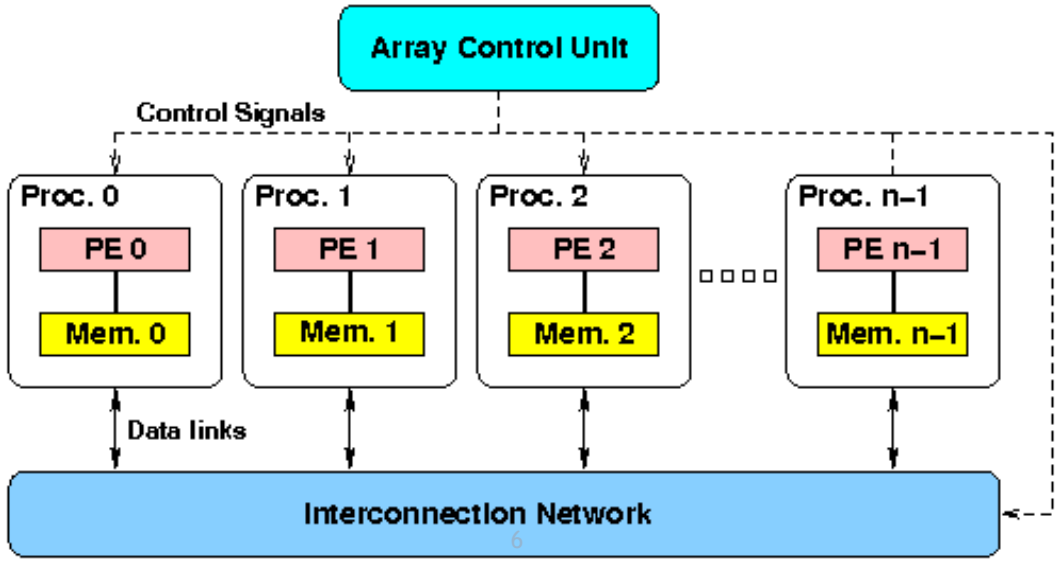
❑ Applications:

Image processing ,

Scientific simulations ,

Matrix computations

,Signal processing



VECTOR PROCESSOR

A **Vector Processor** is a type of SIMD architecture that performs operations on **entire vectors (arrays of data) at once** using **vector registers**, instead of processing individual data elements.

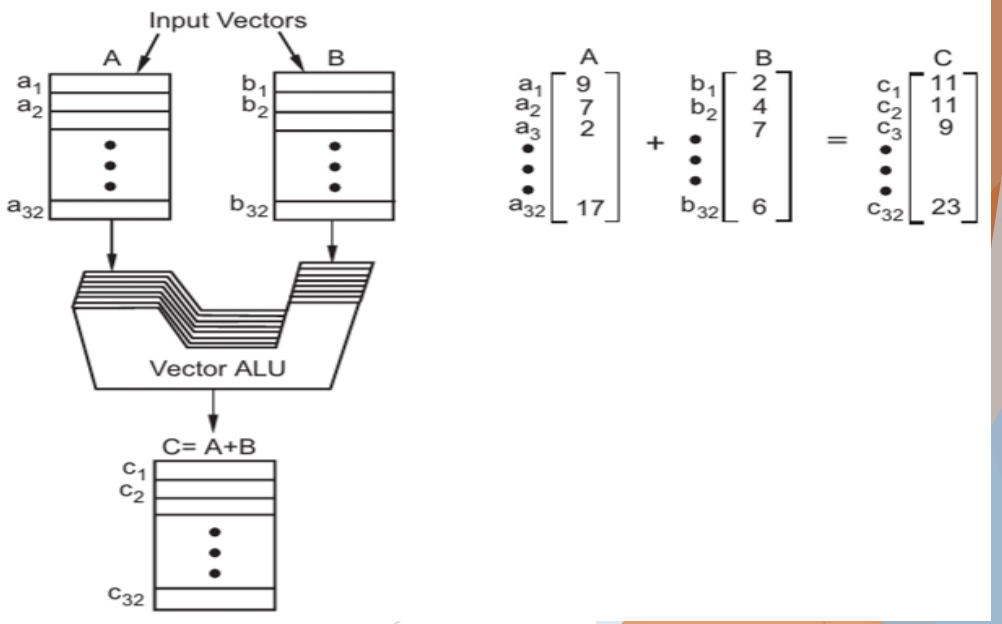
❑ **Main Components :**

•**Control Unit (CU):**Issues instructions for vector operations

•**Vector Registers:**Store multiple data elements (e.g., A1, A2, A3…)

•**Arithmetic Logic Unit (ALU):**Performs operations on entire vectors

•**Memory Unit:**Loads and stores vector data



❑ **Working :**

1.Data is loaded into vector registers

2.Control Unit issues a vector instruction

3.ALU processes all elements simultaneously

4.Result is stored in a result vector

➢ Example: Adding two vectors in a single operation

❑ **Applications**

Scientific computing ,Machine learning ,Image and video processing ,Weather modeling

ARRAY vs VECTOR PROCESSOR

Feature	Array Processor	Vector Processor
Processing Style	Multiple processors (PEs)	Single processor with vector units
Data Handling	Each PE handles one data element	Processes entire vector at once
Hardware Structure	Grid/array of processors	Vector registers + ALU
Control Unit	One CU controls all PEs	One CU controls vector operations
Efficiency	High for large parallel tasks	High for sequential vector operations
Complexity	More hardware complexity	Less hardware complexity
Example Use	Image processing, matrix ops	Scientific computing, ML

SIMD ALGORITHMS

SIMD algorithms are computational methods designed to **apply a single instruction to multiple data elements simultaneously**, exploiting data-level parallelism.

They are widely used in:

•Scientific computing

•Machine learning

•Image & signal processing

❑ Core Idea:

Instead of:

Process one data → then next → then next

SIMD does:

Process multiple data elements at the same time

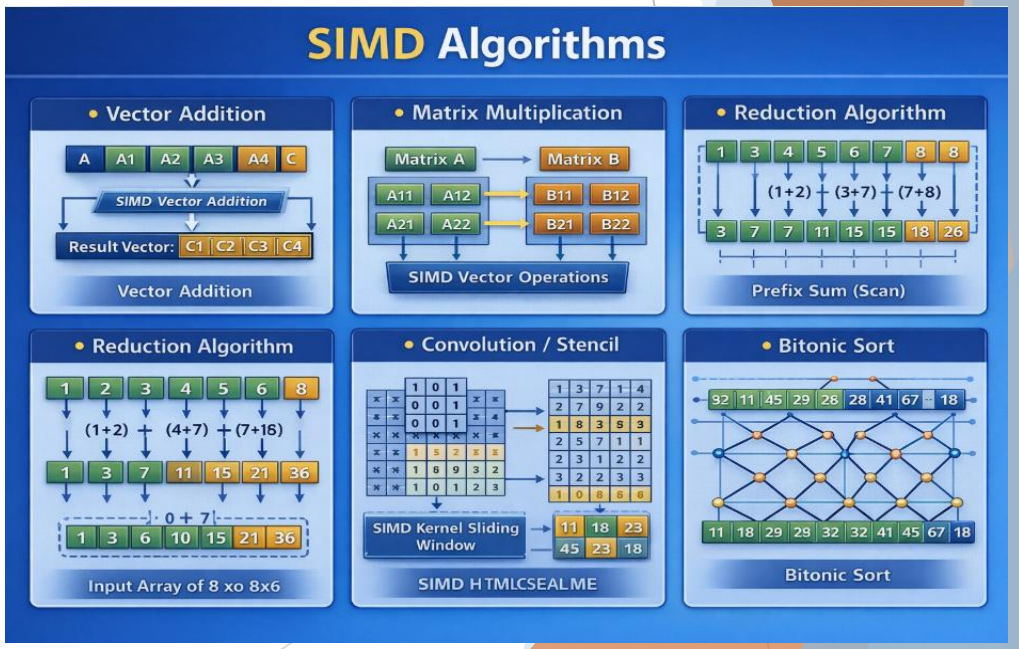
❑ Key Characteristics of SIMD Algorithms :

•Operate on **vectors (arrays of data)**

•Require **uniform operations**

•Minimize instruction overhead

•Use **vector registers / parallel processors**



❑ Types of SIMD Algorithm Patterns:

SIMD algorithms are generally based on these patterns:

1.Vector Addition Algorithms

2.Matrix Multiplication Algorithms

3.Reduction Algorithms

4. Scan (Prefix Sum) Algorithms

5. Stencil / Convolution Algorithms

6. Bitonic Sort

Vector Addition Algorithm (SIMD)

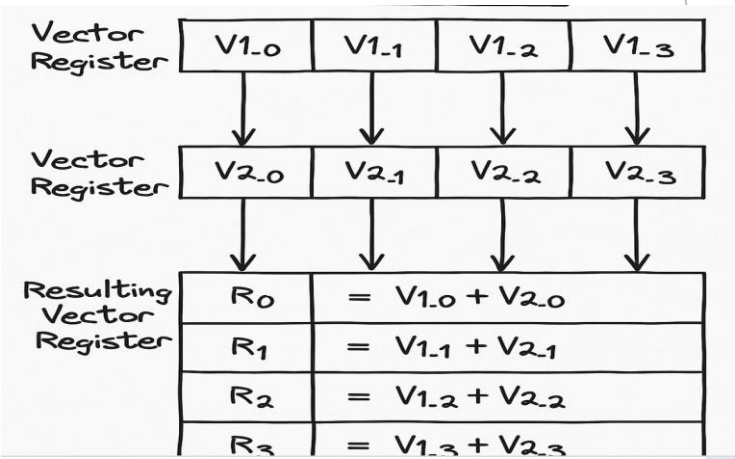
Vector Addition is a basic SIMD algorithm where **two arrays (vectors) are added element-wise using a single instruction applied to multiple data elements simultaneously**.

- **Mathematical Representation:**

$$C[i] = A[i] + B[i]$$

- **SIMD Working:**

1. Load multiple elements of vector **A** into a register
2. Load corresponding elements of vector **B**
3. Perform **parallel addition** using one instruction
4. Store the result in vector **C**



- Example:

$$[A1\ A2\ A3\ A4] + [B1\ B2\ B3\ B4] = [C1\ C2\ C3\ C4]$$

- **Key Advantages:**

✓ Reduces execution time

✓ Uses data-level parallelism

✓ Efficient for large datasets

✓ Minimizes loop overhead

- **Applications:**

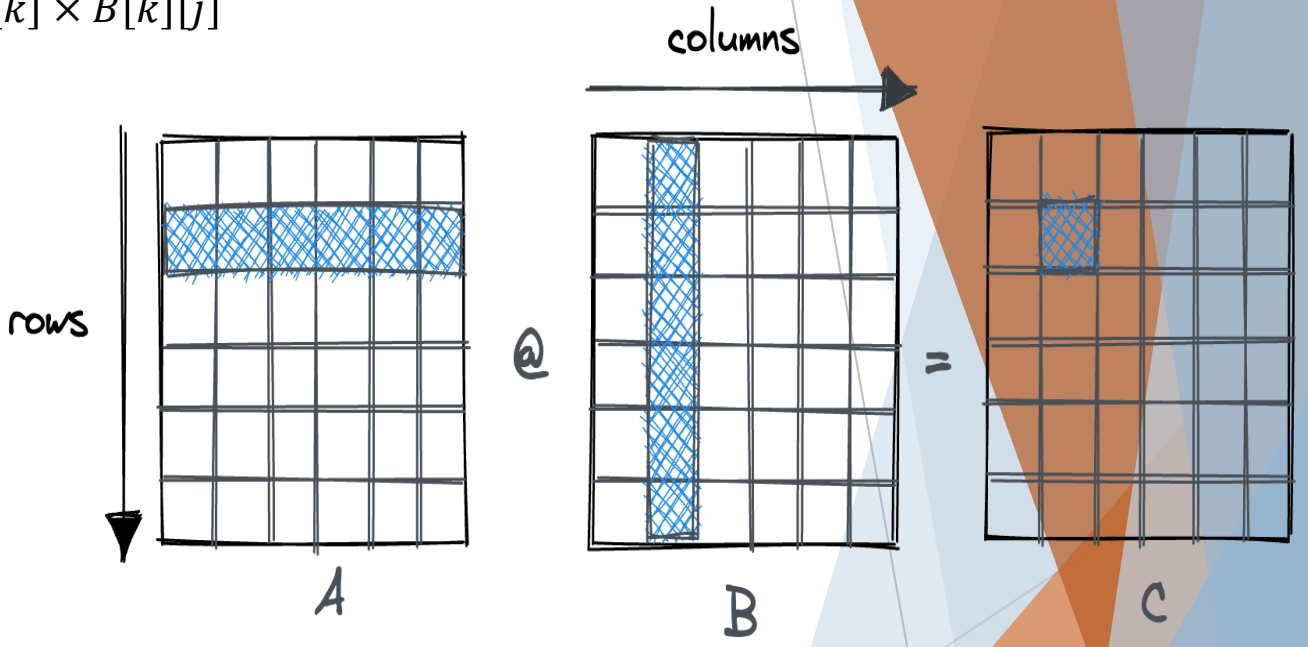
Scientific computations ,Image processing ,Machine learning ,Signal processing

MATRIX MULTIPLICATION

Matrix Multiplication in SIMD is an optimized algorithm where **rows and columns of matrices are processed in parallel using vector operations**, improving computational efficiency.

- **Mathematical Representation:** $C[i][j] = \sum_k A[i][k] \times B[k][j]$
- **SIMD Working:**

1. Load a row of matrix **A** into a vector register
2. Load a column of matrix **B** into another register
3. Perform **parallel multiplication**
4. Apply **vector reduction (sum)** to get final value
5. Store result in matrix **C**



- **Key Advantages**

✓ Faster than scalar multiplication

✓ Efficient for large matrices

✓ Widely used in high-performance computing

- **Applications**

Machine learning (neural networks) ,Graphics rendering ,Scientific simulations ,Data analysis

REDUCTION ALGORITHM

Reduction is a SIMD algorithm used to **combine multiple data elements into a single result** (e.g., sum, minimum, maximum) using parallel operations.

- **Mathematical Representation:** $S = \sum_{i=1}^{n} A[i]$
- **SIMD Working:**

1.Load data into vector registers

2.Perform **pairwise operations** (addition/min/max)

3.Reduce results step-by-step (tree structure)

4.Obtain final single value

- Example:$[1\ 2\ 3\ 4] \rightarrow (1, +2\ 3 + 4) \rightarrow (3, 7) \rightarrow 10$

- **Key Advantages:**

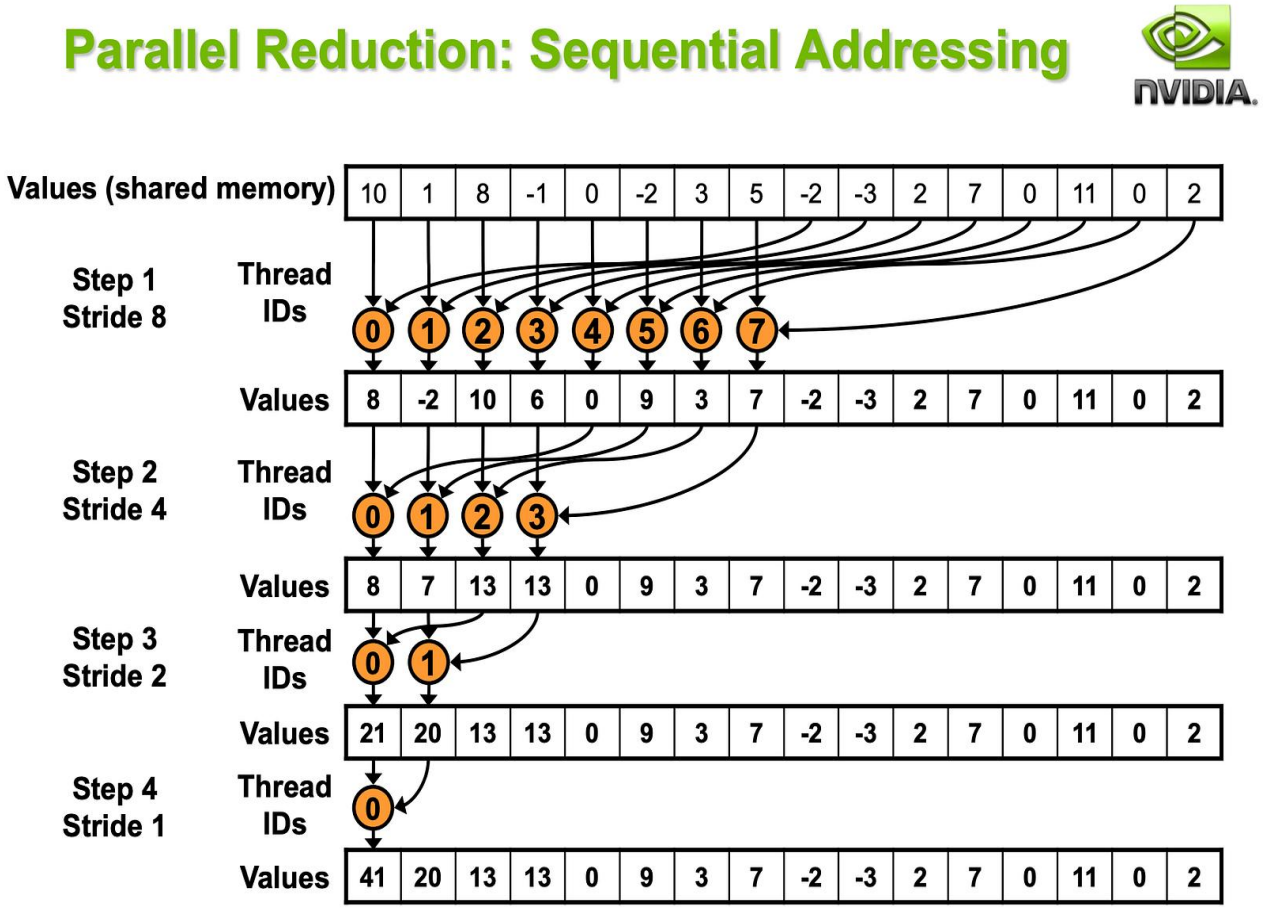
✓ Logarithmic time complexity

✓ Efficient aggregation

✓ Reduces computation steps

- **Applications:**

Sum, min, max operations ,Statistical analysis ,Machine learning



SCAN(PREFIX SUM) Algorithm

Prefix Sum (Scan) is a SIMD algorithm that computes **cumulative sums of elements**, where each output depends on previous elements.

- **Mathematical Representation:** $Output[i] = A[1] + A[2] + \ldots + A[i]$

- **SIMD Working:**

Phase 1: Up-Sweep (Reduce Phase)

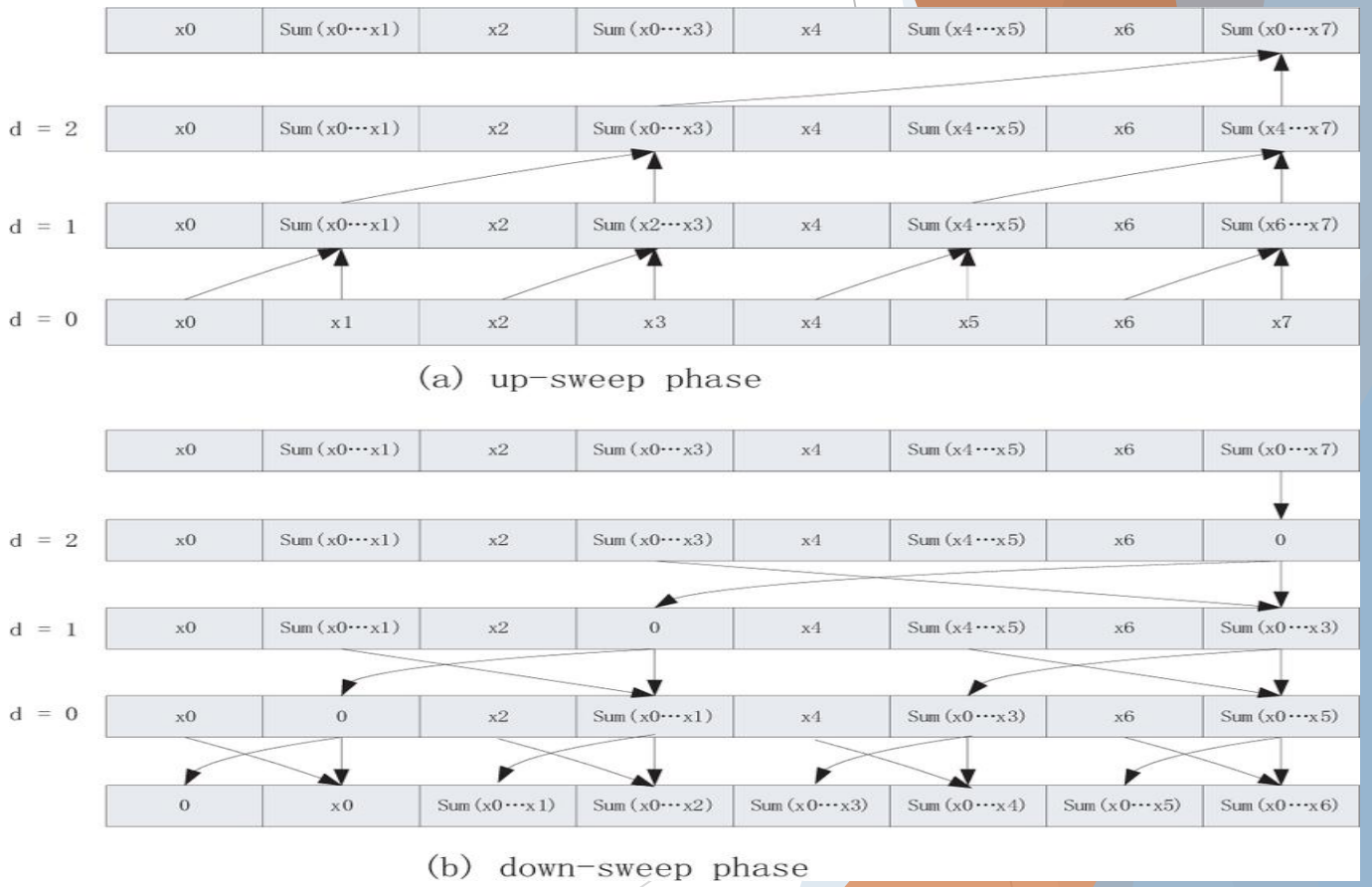
•Build partial sums

Phase 2: Down-Sweep (Distribution Phase)

•Compute final prefix values

➢ **Example :** $Input$: [1 2 3 4]

$Output$: [1 3 6 10]



- **Key Advantages:**

✓ Enables parallel cumulative computation

✓ Efficient for large datasets

✓ Used in many parallel algorithms

- **Applications:**

Parallel sorting ,Graph algorithms ,GPU programming ,Data analysis

Convolution / Stencil Algorithm

Convolution (Stencil) is a SIMD algorithm where a **kernel (filter)** is applied across data (image/signal) to produce transformed output using parallel operations.

- **Mathematical Representation:** $Output[i] = \sum_k K\,ernel[k] \times Input[i+k]$

- **SIMD Working :**

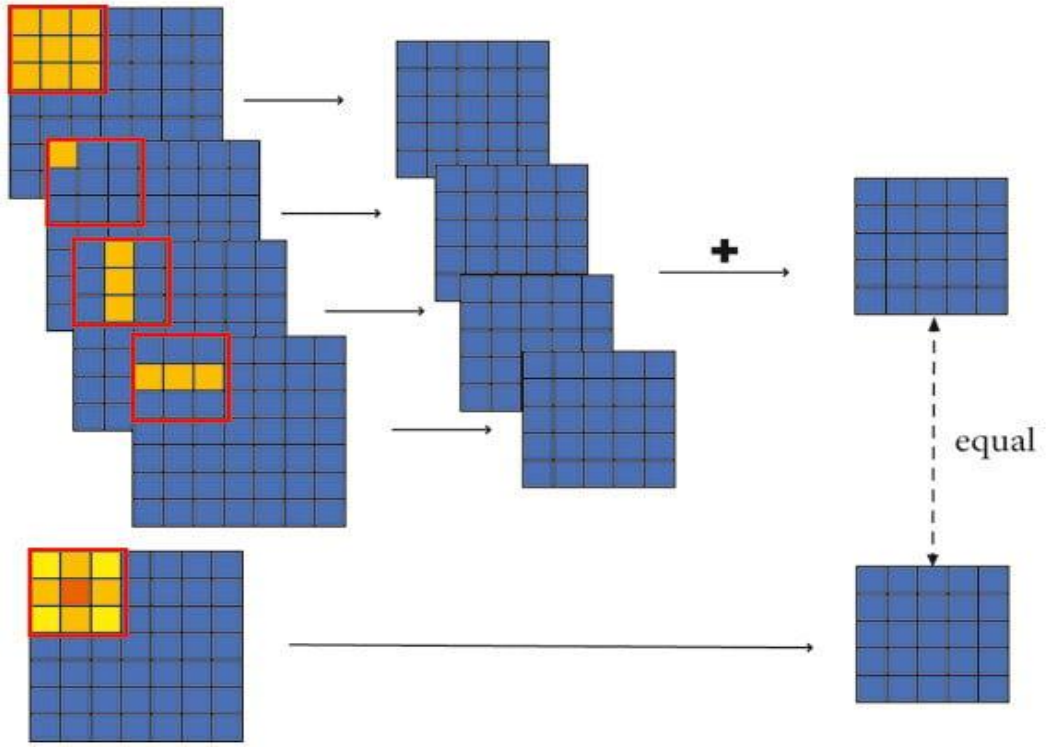
1.Load input data into vector registers

2.Apply **kernel (filter)** across multiple elements simultaneously

3.Perform **parallel multiplication and addition**

4.Slide the window and repeat

➢ Example: Image filtering (blur, edge detection)



- **Key Advantages:**

✓ High-speed filtering

✓ Efficient for large datasets

✓ Exploits data parallelism

- **Applications:**

Image processing (blur, sharpen, edge detection) ,Signal processing ,Convolutional Neural Networks (CNNs)

BITONIC SORT

Bitonic Sort is a parallel sorting algorithm designed for SIMD systems, using **compare-and-swap operations in a fixed pattern**.

- **Key Concept:**

A **bitonic sequence** is a sequence that first increases and then decreases (or vice versa).

- **SIMD Working:**

1.Divide data into sequences

2.Create **bitonic sequences**

3.Perform **compare-and-swap operations in parallel**

4.Merge sequences to get sorted output

- **Key Advantages:**

✓ Highly parallelizable

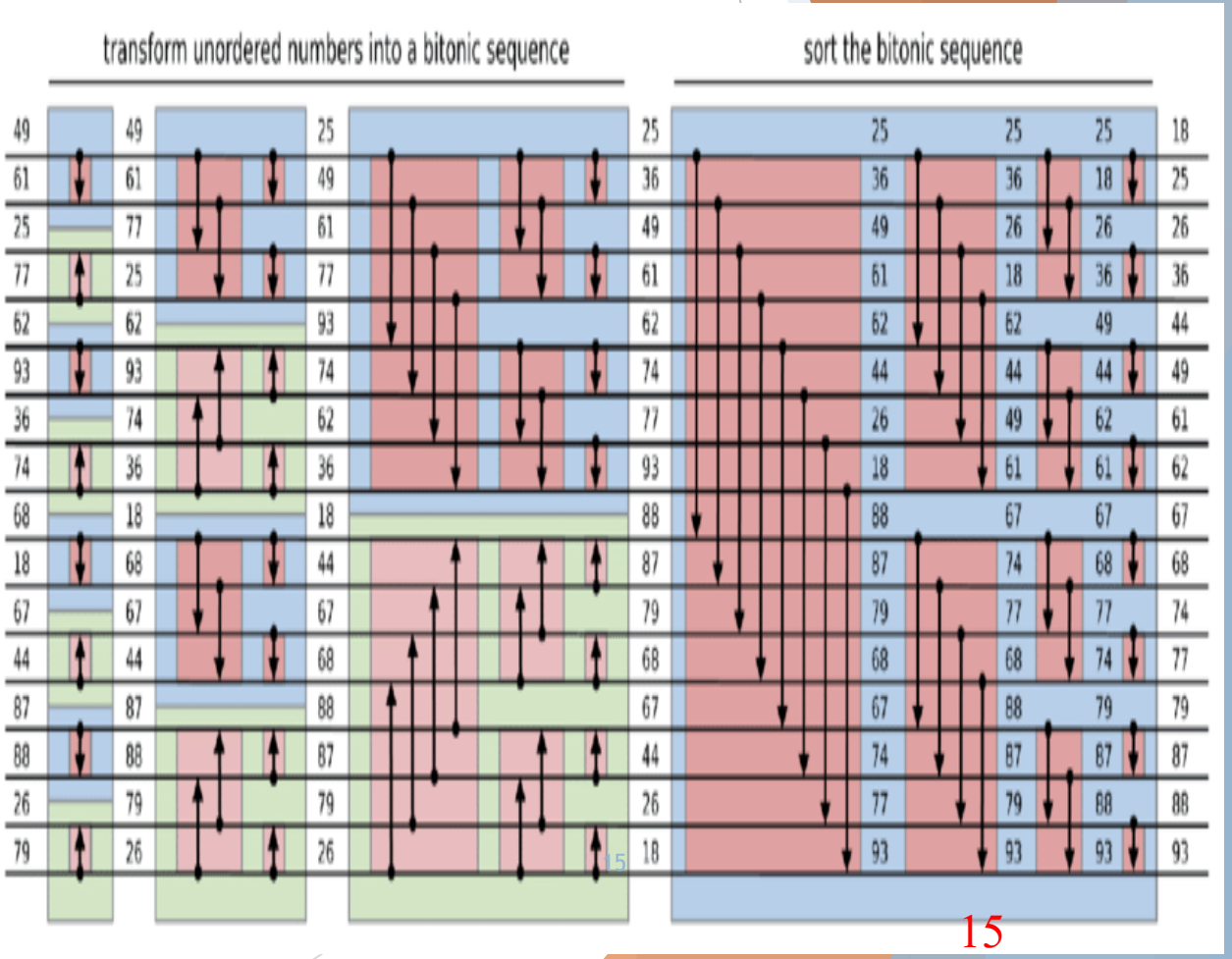
✓ Suitable for SIMD/GPU architectures

✓ Predictable execution pattern

- **Applications:**

GPU-based sorting ,Parallel computing systems

,High-performance computing



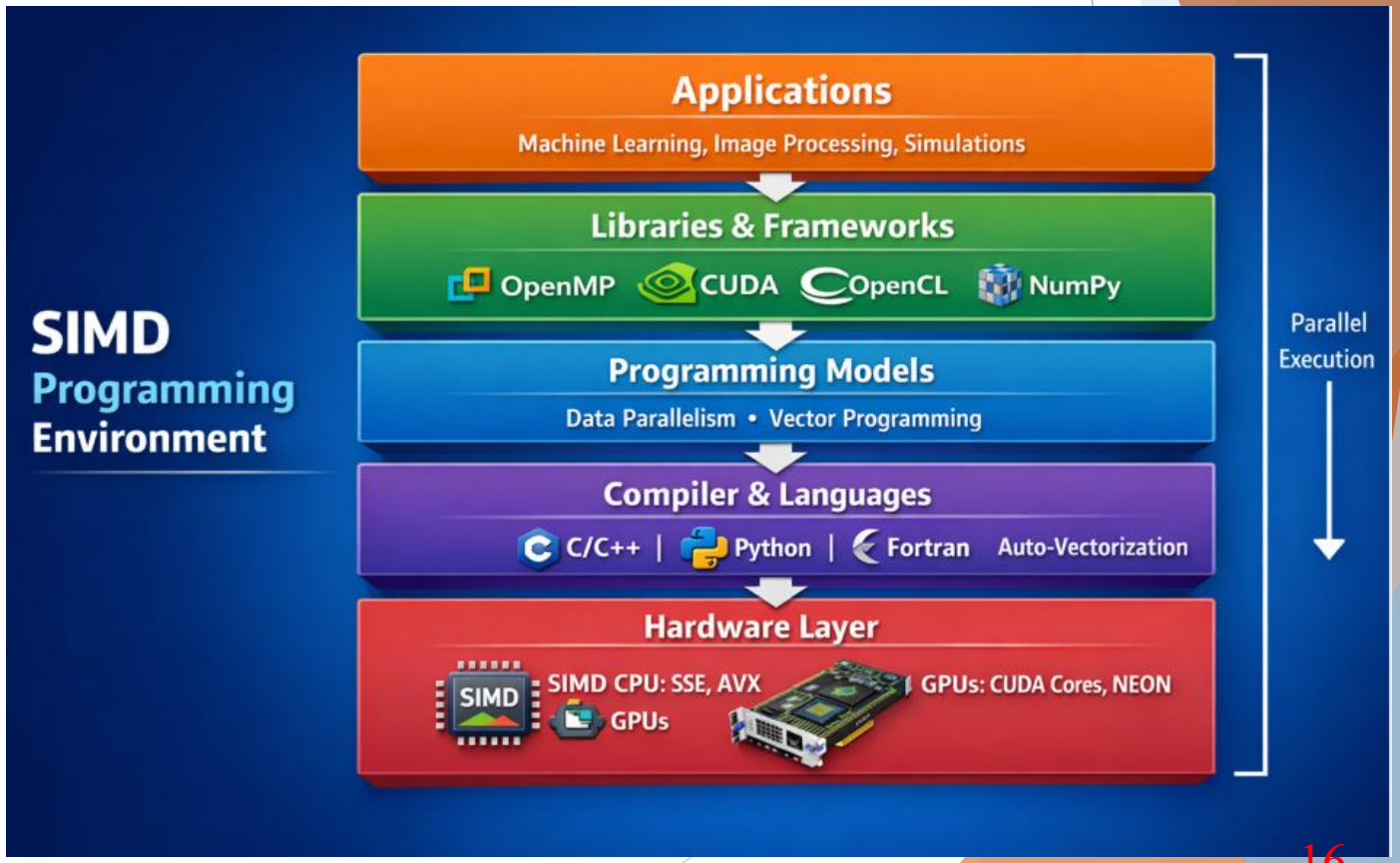
SIMD PROGRAMMING ENVIRONMENT

The **SIMD Programming Environment** refers to the combination of **hardware, software, programming models, and tools** that enable execution of SIMD-based parallel algorithms.

- It provides the platform where **one instruction operates on multiple data elements efficiently**.

❑ **<u>Layers of SIMD Environment</u>**:

1. Hardware Layer
2. Compiler Support
3. Programming Models
4. Libraries & Frameworks
5. Programming Languages



HARDWARE LAYER (Bottom Layer)

This is the **physical execution layer** where actual SIMD operations occur.

- **Components**

•Vector processing units

•SIMD registers

•GPUs with thousands of cores

- **Technologies**

•Intel SSE, AVX

•ARM NEON

•GPUs (CUDA cores)

- **Functions**

•Execute instructions in parallel

•Handle multiple data streams

- **Research Insight:**

➢ Hardware determines the **maximum achievable parallel performance**

COMPILER & LANGUAGE LAYER

This layer translates high-level code into **SIMD instructions**.

❑ **Languages**

•C / C++ (intrinsics, vector extensions)

•Python (NumPy, Tensor libraries)

•Fortran

❑ **Compiler Functions**

•**Auto-vectorization** (automatic SIMD conversion)

•Instruction optimization

•Memory alignment handling

❑ **Examples**

•GCC

•Clang

•Intel Compiler

❑ **Research Insight:**

➢ Compiler efficiency directly impacts SIMD performance

PROGRAMMING MODELS LAYER

Defines **how parallelism is expressed** in a program.

❑ **Types of Models:**

✓ **Data Parallel Model**

•Same operation on multiple data

•Basis of SIMD

✓ **Vector Programming Model**

•Uses vector instructions and registers

❑ **Key Functions:**

•Structure parallel computation

•Define execution strategy

❑ **Research Insight:**

➢ Programming models bridge **human logic and machine execution**

LIBRARIES & FRAMEWORK LAYER

Provides **high-level tools and APIs** that allow developers to use SIMD without handling low-level hardware details.

- **Key Tools**

•OpenMP → CPU-based parallelism

•OpenCL → Cross-platform

•NumPy → Implicit SIMD

- **Functions**

•Provide **parallel constructs**

•Simplify SIMD programming

•Optimize performance automatically

- **Research Insight**

➢ These frameworks hide complexity and **enable scalable parallel computing**

APPLICATION LAYER (Top Layer)

This is the **end-user layer** where SIMD is actually applied to solve real-world problems.

❑ **Key Functions**

•Defines the **problem and computation requirements**

•Uses SIMD indirectly through libraries/frameworks

❑ **Examples of Applications**

•Machine Learning (neural networks, training)

•Image Processing (filters, edge detection)

•Scientific Simulations (weather, physics)

•Signal Processing

❑ **Research Point**

➢ SIMD improves performance in applications that require **repetitive operations on large datasets**

APPLICATIONS OF SIMD

SIMD is widely used in:

🧪 **Scientific Computing:**

•Simulations

•Numerical methods

🎮 **Graphics & Gaming:**

•Rendering

•Physics engines

🤖 **Machine Learning:**

•Matrix operations

•Neural networks

🖼 **Image & Video Processing:**

•Filters

•Compression

ADVANTAGES and LIMITATIONS of SIMD

- **Advantages of SIMD:**

✓ High performance

✓ Efficient for large datasets

✓ Reduced instruction overhead

✓ Energy efficient

- **Limitations of SIMD:**

❌ Not suitable for branching-heavy tasks

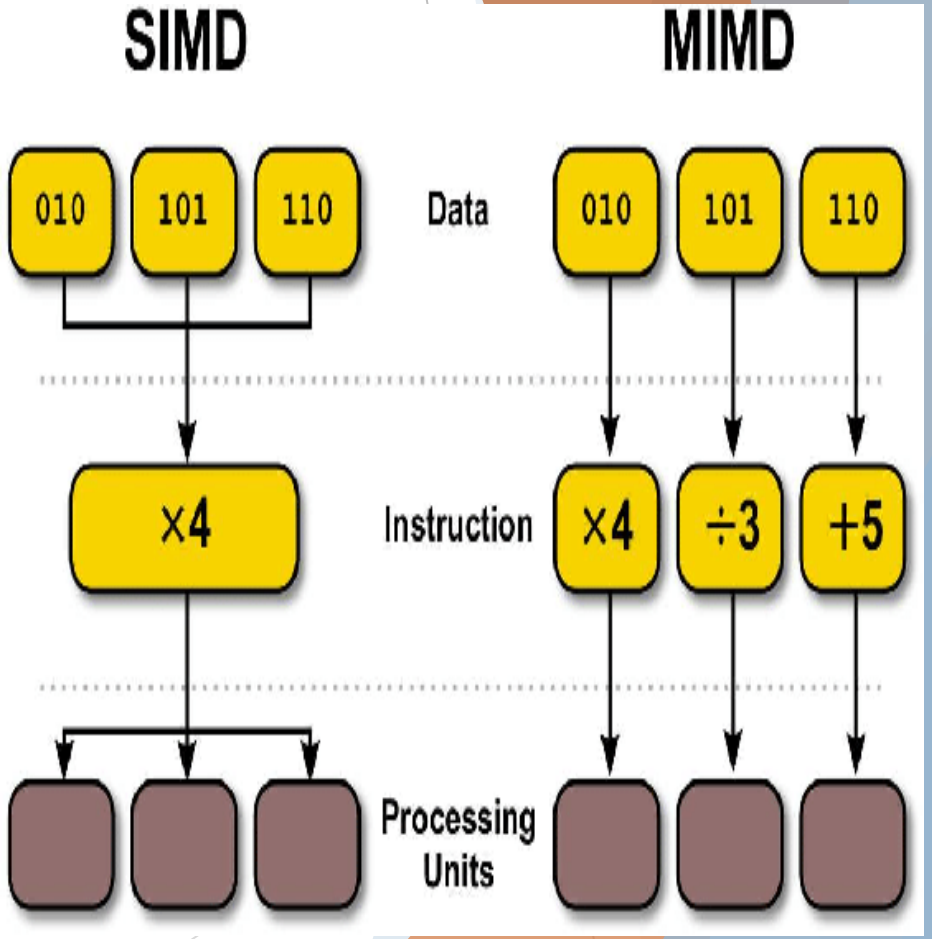
❌ Memory alignment issues

❌ Limited flexibility

❌ Requires specialized hardware

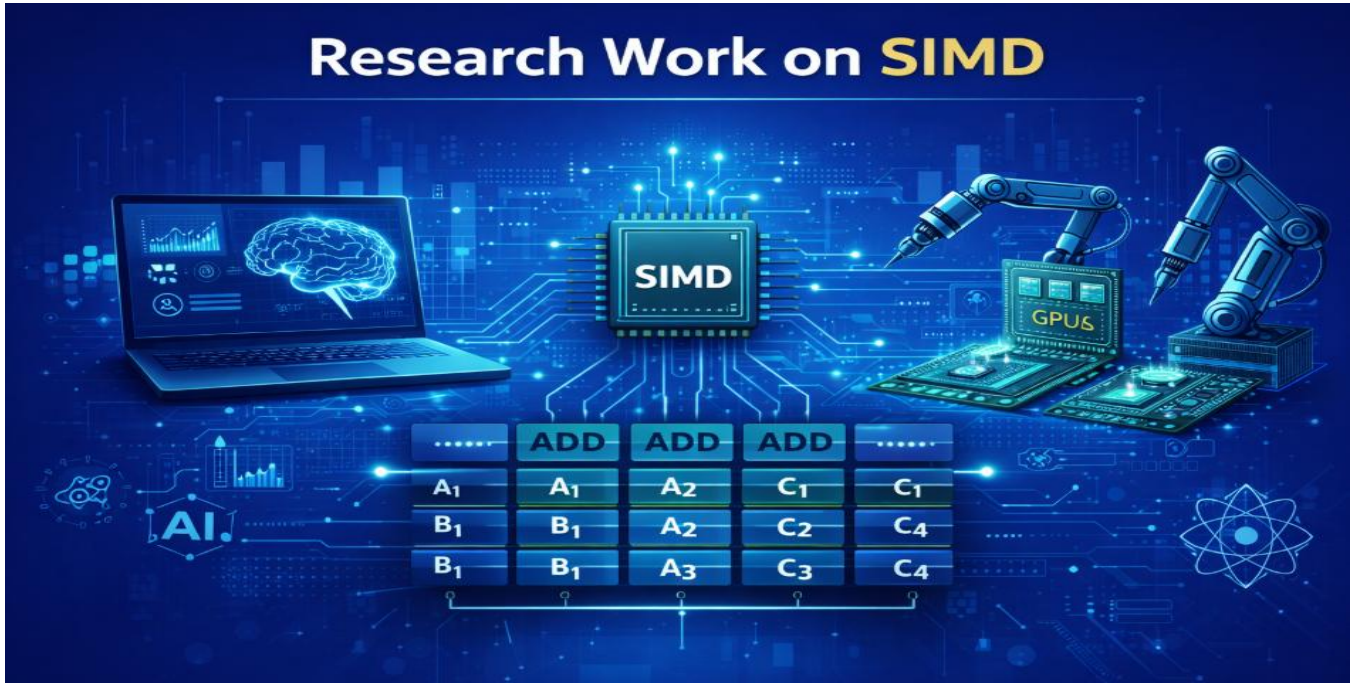
COMPARISON – SIMD vs MIMD Architecture

Feature	SIMD	MIMD
Instruction Stream	Single	Multiple
Processing Style	Same operation on all data	Different operations on different data
Control Unit	One control unit	Multiple control units
Flexibility	Low	High
Complexity	Simple architecture	Complex architecture
Performance	High for uniform tasks	High for complex tasks
Scalability	Limited	Highly scalable
Examples	GPUs, Vector processors	Multi-core CPUs, distributed systems



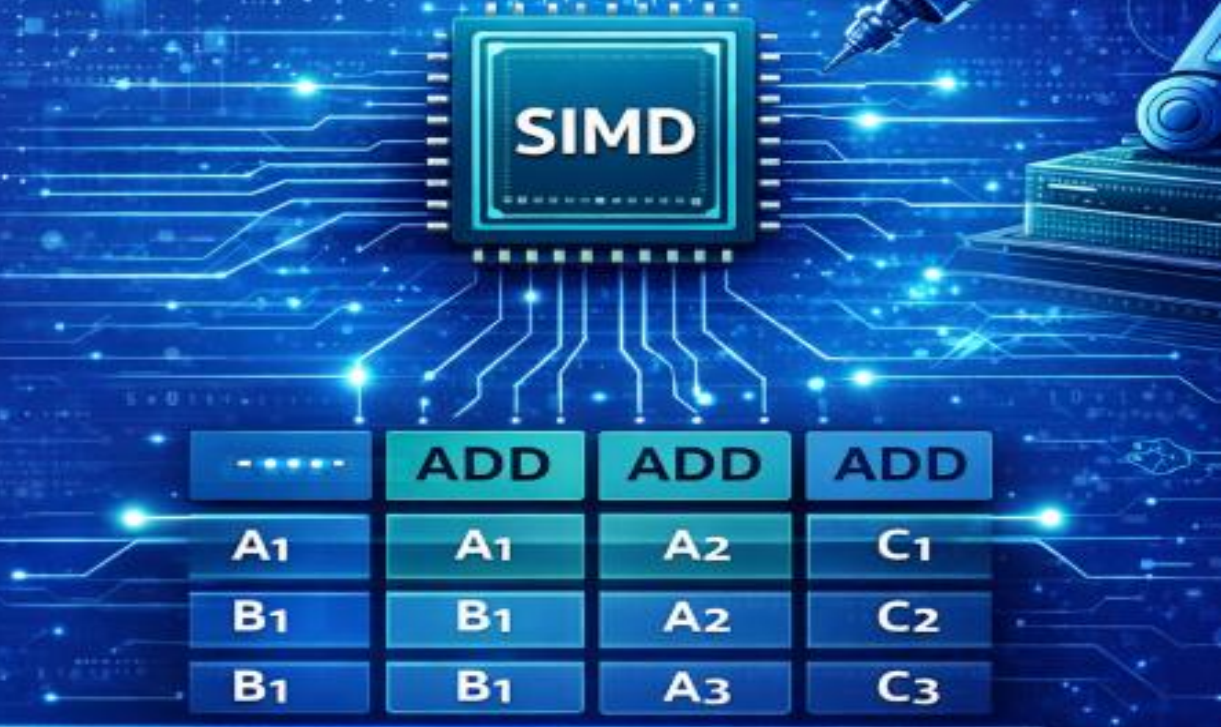
RESEARCH IMPORTANCE

- Widely used in **AI, machine learning, graphics, and scientific simulations**
- Essential for handling **large-scale data-intensive applications**
- Forms the foundation of modern **GPU and vector processing technologies**



Conclusion

- SIMD enables **parallel processing** of multiple data using a single instruction
- Improves **performance, speed, and efficiency** in large-scale computations
- Widely used in **AI, image processing,** and **scientific** applications



SIMD is a key technology for modern high-performance computing through **data-level parallelism.**

"SIMD helps computers process large amounts of data faster by doing many operations at the same time."

THANK YOU